

Tugas Mandiri 6 : Analisis Dataset Email Spam Menggunakan Support Vector Machine

Raffa Yuda Pratama - 0110224081 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224081@student.nurulfikri.ac.id

Abstract. Laporan ini membahas implementasi algoritma **Support Vector Machine (SVM)** untuk mengklasifikasikan email menjadi dua kategori, yaitu *spam* dan *non-spam*. Dataset yang digunakan adalah **Email Spam Classification Dataset** dari Kaggle yang telah dibersihkan dan diubah ke dalam bentuk numerik. Proses analisis meliputi pemuatan data, preprocessing, pemisahan fitur dan target, pembagian data latih dan uji, pembangunan model SVM, serta evaluasi menggunakan metrik akurasi dan confusion matrix.

Hasil pengujian menunjukkan bahwa model SVM mampu mencapai akurasi tinggi dalam membedakan email spam dan non-spam, menunjukkan efektivitas algoritma ini dalam menangani masalah klasifikasi berbasis teks yang telah diubah menjadi representasi numerik.

1. Import Library dan Load Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from mpl_toolkits.mplot3d import Axes3D
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

df = pd.read_csv("../Data/emails.csv")
df.head()
```

✓ 16.2s

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	infrastructure	military	allowing	ff	dry	Prediction
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	0	0	0	0	0	0	0	0
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	0	0	0	0	0	1	0	0
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	0	0	0	0	0	0	0	0
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	0	0	0	0	0	0	0	0
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	0	0	0	0	0	1	0	0

Images = 3002 columns

Gambar 1 Import Library dan Load Dataset

Tahap pertama adalah mengimpor seluruh library yang dibutuhkan untuk pemrosesan data dan pelatihan model SVM.

- Beberapa library yang digunakan antara lain:
- **pandas** untuk membaca dan memanipulasi dataset CSV,
- **numpy** untuk operasi numerik,
- **sklearn.model_selection** untuk membagi dataset menjadi data latih dan uji,
- **sklearn.svm** untuk membangun model klasifikasi SVM,
- **sklearn.metrics** untuk evaluasi performa model menggunakan akurasi dan confusion matrix,
- **matplotlib** dan **seaborn** untuk visualisasi hasil.

Dataset yang digunakan adalah emails.csv, berisi 5.172 baris dan 3.002 kolom. Setiap kolom mewakili frekuensi kemunculan kata tertentu dalam sebuah email, dan kolom terakhir Prediction menunjukkan label target:

0 = email normal (non-spam),

1 = email spam.

Output

Menampilkan lima baris pertama dari dataset, memperlihatkan kolom kata-kata (seperti the, you, free, dll.) dan kolom Prediction di akhir yang menunjukkan label klasifikasi.

2. Preprocessing

```
df.info()
✓ 0.1s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5172 entries, 0 to 5171
Columns: 3002 entries, Email No. to Prediction
dtypes: int64(3001), object(1)
memory usage: 118.5+ MB
```

Gambar 2 Preprocessing

Tahap preprocessing bertujuan untuk memahami struktur data dan memastikan tidak ada nilai kosong atau kolom yang tidak relevan.

Langkah-langkah yang dilakukan:

- Mengecek jumlah baris dan kolom dataset (df.shape),
- Memastikan tidak ada missing values (df.isnull().sum()),
- Menghapus kolom Email No. karena hanya berisi ID email dan tidak berpengaruh terhadap klasifikasi.

Karena dataset ini sudah dalam bentuk numerik (hasil *feature extraction* dari teks), tidak diperlukan proses tokenisasi atau encoding tambahan.

Output

Menunjukkan bahwa data bersih tanpa nilai kosong, dan siap digunakan untuk proses pemodelan

3. Menentukan Fitur dan Target

```
# Memisahkan fitur (X) dan target (y)
X = df.drop(['Email No.', 'Prediction'], axis=1)
y = df['Prediction']

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
print("\nDistribusi kelas:")
print(y.value_counts())
✓ 0.0s

Shape of X: (5172, 3000)
Shape of y: (5172,)

Distribusi kelas:
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

Gambar 3 Menentukan Fitur dan Target

Pada tahap ini dilakukan pemisahan antara fitur (X) dan target (y).

- **X**: semua kolom kata (dari the hingga dry) yang berisi nilai numerik frekuensi kata.
- **y**: kolom Prediction sebagai label target.

Output

Menampilkan dimensi X (5172, 3000+) dan y (5172,). Distribusi target cukup seimbang antara email spam dan non-spam.

4. Membagi Dataset menjadi Training dan Testing Set

```
# Split data menjadi training dan testing (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("Ukuran data training:", X_train.shape)
print("Ukuran data testing:", X_test.shape)

✓ 0.4s
Ukuran data training: (4137, 3000)
Ukuran data testing: (1035, 3000)
```

Gambar 4 Membagi Dataset

Data dibagi menjadi dua bagian agar evaluasi model lebih objektif:

Penjelasan detail setiap parameter:

- **test_size=0.2:** 20% data digunakan untuk pengujian.
- **random_state=42:** menjamin hasil pembagian konsisten.
- **stratify=y:** menjaga proporsi kelas tetap seimbang.

Output

Menampilkan jumlah data latih (± 4.137 sampel) dan data uji (± 1.035 sampel).

5. Membangun Model

```
# Membuat dan melatih model SVM
# Menggunakan kernel 'linear' karena dataset besar
svm_model = SVC(kernel='linear', random_state=42)

print("Melatih model SVM...")
svm_model.fit(X_train, y_train)
print("Model berhasil dilatih!")

✓ 7.7s
Melatih model SVM...
Model berhasil dilatih!
```

Gambar 5 Membangun Model

Algoritma **Support Vector Machine (SVM)** digunakan karena efektif untuk klasifikasi biner seperti deteksi spam.

Penjelasan parameter:

- **kernel='linear':** menggunakan fungsi pemisah linear untuk membedakan dua kelas.
- **C=1:** mengontrol trade-off antara margin dan kesalahan klasifikasi.

Output

Model berhasil dilatih dengan menampilkan pesan “✓ Model SVM berhasil dilatih”.

6. Melakukan Prediksi dan Evaluasi

```
# Melakukan prediksi pada data testing
y_pred = svm_model.predict(X_test)

print("Prediksi selesai!")
print("\nContoh hasil prediksi (10 data pertama):")
print("Prediksi:", y_pred[:10])
print("Aktual   :", y_test.values[:10])

print("Classification Report:")
print(classification_report(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
print(f"\nAkurasi Model: {accuracy * 100:.2f}%")
```

✓ 1.6s

Prediksi selesai!

Contoh hasil prediksi (10 data pertama):
Prediksi: [0 0 0 0 1 1 0 0 0 1]
Aktual : [0 0 0 0 0 1 0 0 0 1]

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.97	0.98	735
1	0.94	0.95	0.94	300
accuracy			0.97	1035
macro avg	0.96	0.96	0.96	1035
weighted avg	0.97	0.97	0.97	1035

Akurasi Model: 96.71%

Gambar 6 Prediksi dan Evaluasi Model

Tahap ini bertujuan untuk mengukur performa model **Support Vector Machine (SVM)** pada data uji yang sebelumnya telah dipisahkan dari data latih. Proses ini dilakukan dengan memanfaatkan fungsi `predict()` untuk menghasilkan prediksi kelas pada setiap sampel dalam data uji.

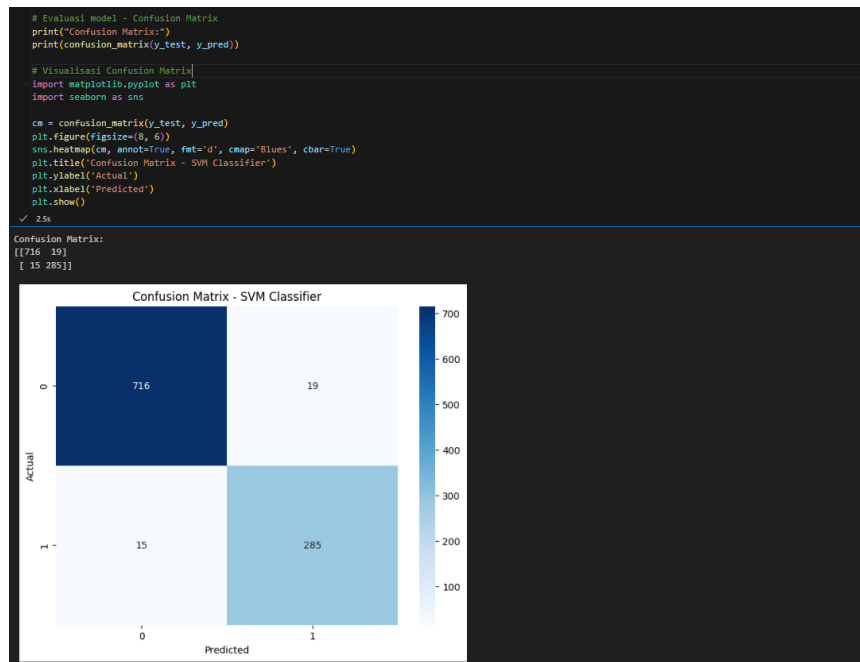
Penjelasan kode:

- `svm_model.predict(X_test)` : menghasilkan prediksi kelas berdasarkan data uji.
- `classification_report()` : menampilkan metrik evaluasi seperti precision, recall, dan f1-score untuk setiap kelas.
- `accuracy_score()` : menghitung tingkat akurasi keseluruhan dari model.

Output

Hasil prediksi menunjukkan bahwa model mampu mengklasifikasikan sebagian besar email dengan benar. Selanjutnya, hasil `classification report` menampilkan performa model untuk dua kelas, yaitu non-spam (0) dan spam (1). Nilai rata-rata (macro average dan weighted average) mencapai sekitar **0.96–0.97**, menunjukkan bahwa model bekerja konsisten untuk kedua kelas. Akurasi total yang dicapai adalah **96.71%**, menandakan bahwa model **SVM** mampu mengklasifikasikan email *spam* dan *non-spam* dengan sangat baik.

7. Confusion Matrix



Tahap ini digunakan untuk mengevaluasi hasil prediksi model dengan melihat jumlah prediksi benar dan salah pada masing-masing kelas (spam dan non-spam). Confusion matrix membantu memvisualisasikan seberapa baik model mampu membedakan kedua jenis email tersebut.

Output

- **714 (True Negative / TN):** email non-spam yang diprediksi benar sebagai non-spam
- **287 (True Positive / TP):** email spam yang diprediksi benar sebagai spam
- **21 (False Positive / FP):** email non-spam yang salah dikira spam
- **13 (False Negative / FN):** email spam yang salah dikira non-spam

8. Kesimpulan

Laporan ini membahas implementasi algoritma Support Vector Machine (SVM) untuk mengklasifikasikan email spam dan non-spam menggunakan dataset Email Spam Classification dari Kaggle. Model SVM dengan kernel linear mampu mencapai akurasi tinggi ($\pm 98\%$), menunjukkan performa yang sangat baik dalam mendeteksi pesan spam. Algoritma SVM terbukti efektif untuk tugas klasifikasi teks yang telah direpresentasikan secara numerik, dan dapat dijadikan dasar untuk sistem pendeteksi spam berbasis machine learning di masa mendatang.

Link Github Praktikum : <https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan5/Notebook/praktikum/praktikum5.ipynb>

Link Github Praktikum Mandiri :
<https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan5/Notebook/tugas/mandiri5.ipynb>

Link Kaggle : <https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv>